UT2 · Ejemplo 10
Sensores repositorio y Composer

Desarrollo Web en Entorno Servidor · 2.º DAW · Curso 2026/2027
Departamento de Informática · IES Ramón Arcas Meca · Lorca

1. Objetivo

Completa un monitor de sensores con Composer y namespaces. El modelo valida lecturas, un repositorio conserva los objetos y un servicio consulta alertas. El nivel se calcula a partir de la lectura actual; no se cambia con acciones de inicio o finalización.

2. Preparar Composer

2.1 Estructura facilitada

Abre la carpeta completa del ejemplo. Contiene composer.json, bin/sensores.php y src con Modelo, Repositorio y Servicio. El enum, las consultas del modelo y los constructores están preparados. Completa los TODO; no cambies sus firmas.

1. En composer.json, añade autoload PSR-4: Aula\Sensores\ se corresponde con src/. Conserva name, type y require.

2. Añade un script llamado sensores que ejecute @php bin/sensores.php. En JSON, cada barra inversa de un namespace se escribe dos veces.

3. Desde la terminal del editor situada junto a composer.json, ejecuta estas órdenes. Utiliza el PHP de clase con mbstring y Composer.

```
composer validate --no-check-publish
composer dump-autoload
composer sensores
```

La entrada ya incluye vendor/autoload.php. Las clases se cargarán por Composer; no añadas require manuales de clases. No se necesitan paquetes externos. Tras configurar Composer, los TODO mostrarán qué falta implementar.

3. Datos y nivel del sensor

3.1 Datos iniciales

Id	Sala	Umbral	Lectura
91	Robótica	60	55
92	Diseño	50	50
93	Robótica	70	null
94	Redes	80	0

Las lecturas son enteros de 0 a 100. null significa que todavía no hay lectura; cero es una medida válida. Con lectura mayor o igual al umbral, nivel devuelve Nivel::Alerta; por debajo devuelve Nivel::Normal. Sin lectura devuelve null.

3.2 Actualizar y consultar

1. Completa Sensor::registrar: valida de 0 a 100 antes de guardar. Lanza InvalidArgumentException fuera del rango y conserva la lectura anterior.

2. Completa Sensor::nivel. Obtén el nivel a partir de lectura y umbral cada vez; no guardes otra propiedad para él.

4. Repositorio y servicio

4.1 Encontrar los mismos objetos

1. Completa buscar en SensorRepositoryMemoria: recorre los objetos, compara el id y devuelve el encontrado o null. El id no es una posición del array.

2. Conserva las instancias recibidas. No reconstruyas sensores en buscar ni en todos. La interfaz SensorRepository declara las dos operaciones.

3. Completa MonitorService::registrar. Busca el id mediante el repositorio; si no existe, lanza OutOfBoundsException. Si existe, delega la validación y el cambio en Sensor::registrar.

4.2 Alertas y resumen

1. En alertas, obtén todos los sensores del repositorio. Filtra con array_filter y una closure con use, y reindexa con array_values.

2. Conserva solo Nivel::Alerta. Si sala no está vacía, exige igualdad tras trim y minúsculas UTF-8; una sala vacía admite todas.

3. En resumen, utiliza array_reduce con medidos, sinLectura y alertas inicializados a cero. Cada sensor cuenta como medido o sinLectura; además puede contar como alerta.

El servicio depende de la interfaz, no de la implementación en memoria. Las reglas de rango y de nivel pertenecen al modelo. Solo la entrada imprime o captura errores.

5. Resultados y objetos compartidos

Momento	Resultado
Inicio	medidos 3; sinLectura 1; alertas 1; id 92
Tras las cinco operaciones	medidos 4; sinLectura 0; alertas 2; ids 91,93
Filtro ROBÓTICA al final	91,93
Repositorio vacío	Las tres cifras a cero

La entrada registra 60 en 91, 70 en 93 y 10 en 92. Después rechaza 101 en 94 y el id 99. El sensor 94 conserva cero. La lista antigua de alertas mantiene el id 92, pero ese mismo objeto muestra ahora nivel normal: su pertenencia no se recalcula sola.

Identidad debe mostrar true y el id ausente, No existe. La solución completa está en la carpeta homónima de UT2/soluciones/Ejemplos_completos. Genera también allí el autoloader antes de ejecutar composer sensores.